UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
Facultad de Ciencias 2026
Introducción a Ciencias de la Computación

REPORTE — PRÁCTICA 6

Entrada de datos y cierre de la Cabaña del Misterio

Nombre: López Pérez Luis Ángel
Número de cuenta: 324243174
Sistema operativo: Linux
Distribución: Ubuntu
Fecha: **31 / 10 / 2026**

Parte 1: Conociendo Scanner

Actividad 1.1 - Dardos a la medida

Investigación en la documentación de Scanner.

a) ¿Qué hace cada método y qué regresa?

nextInt() lee el siguiente dato que escribe el usuario, lo interpreta como un número entero y lo regresa como un int. hasNextInt() solo revisa, sin leer nada, si el siguiente dato se puede interpretar como un int y regresa un boolean: true si se puede y false si no.

b) ¿Qué pasa si se usa nextInt() y el usuario escribe algo que no es un entero?

El programa lanza una excepción (InputMismatchException) y truena. Además, el dato incorrecto no se consume: sigue en el búfer de entrada, así que un ciclo que vuelva a llamar a nextInt() volvería a fallar con el mismo dato.

c) ¿Cómo ayuda hasNextInt()?

Se usa antes de nextInt() como una revisión previa. Si regresa true, es seguro leer el entero; si regresa false, el programa puede avisar del error y descartar el dato incorrecto sin que se lance ninguna excepción.

Estrategia para actualizar el código. Copié EstimacionPi.java de la Práctica 5 como PiInteractivo.java y reemplacé la cantidad de dardos escrita en el código por una lectura con Scanner. Usé un ciclo while controlado por una variable booleana que indica si la entrada ya es correcta. En cada vuelta el programa pide la cantidad de dardos y revisa con hasNextInt():

1. Si el dato es un entero, lo lee con nextInt(). Si es mayor que cero, la bandera cambia y el ciclo termina. Si es cero o negativo, imprime "Error: El número de dardos debe ser mayor a 0." y vuelve a pedirlo.
2. Si el dato no es un entero, lo descarta con sc.next() (para no quedarme en un ciclo infinito y explotar) e imprime un mensaje que incluye lo que escribió el usuario, por ejemplo "Error: 'dorito' no es un número entero válido.".

Un número demasiado grande, como 9999999999999999, no cabe en un int, por lo que hasNextInt() regresa false y se rechaza igual que un texto. El resto del programa (lanzar los dardos con Math.random(), contar los que caen dentro del círculo y calcular 4.0 × dentro / dardos) se quedó igual que en la Práctica 5.

Escenarios de ejecución.

1. Entrada válida directa**:** Escribí 3000 dardos. El programa mostró 2385 dardos dentro del círculo y una estimación de de 3.180000.
2. Entradas inválidas antes de una válida**:** El programa rechazó el texto dorito, el número 9999999999999999 (fuera del rango de un entero) y el número negativo -89, y explicó el error en cada caso. Después acepté 1 dardo: cayó dentro del círculo y la estimación fue 4.000000. En una ejecución aparte también probé 0.1, que se rechazó como número entero no válido.

Actividad 1.2 - Decimal a binario interactivo

Estrategia para programar BinarioInteractivo.java. El programa tiene tres partes dentro de un ciclo principal:

1. **Lectura y validación.** Un ciclo while pide "Ingresar un número entero no negativo" hasta que la entrada sea correcta. Con hasNextInt() reviso si lo escrito es un entero. Si lo es, lo leo con nextInt() y compruebo que sea mayor o igual que cero; si es negativo, imprimo "Error: El número no puede ser negativo.". Si no es un entero, lo descarto con sc.next() e imprimo "Error: 'texto' no es un número entero válido.".
2. **Conversión.** Copio el número en una variable temporal y uso un while mientras sea mayor que cero. En cada vuelta calculo el residuo de dividir entre 2 con %, lo pego al inicio de un String (binario = residuo + binario) y dividir el número entre 2 con la división entera. Así los residuos quedan en el orden correcto, sin arreglos ni Integer.toBinaryString. El caso del 0 se maneja aparte: el String se inicializa en "0" y no se entra al ciclo.
3. **Resultado.** Imprimo "El número N en binario es: ...".

Estrategia para repetir el programa. Todo lo anterior está dentro de un ciclo while controlado por una bandera booleana continuar, que empieza en true. Al terminar la conversión, el programa pregunta "¿Deseas convertir otro numero? (s/n)" y lee la respuesta con sc.next().charAt(0). Si el carácter es s o S, la bandera sigue en true y el programa vuelve a empezar. Con cualquier otra respuesta, la bandera cambia a false, se imprime "¡Hasta luego!" y el programa termina.

Casos de prueba.

1. **Ejecución sencilla.** Entrada: 65. Salida: "El numero 65 en binario es: 1000001". Respondí n y el programa se despidió.
2. **Entradas inválidas.** Entrada: -2. El programa imprimió "Error: El numero no puede ser negativo." y volvió a pedir el número. Después escribí 02, que Java interpreta como el entero 2, y el programa mostró 10. Los textos y los números con decimales no se leen como enteros, por lo que hasNextInt() regresa false y entran en la misma rama de error que muestra el texto escrito.
3. **Varias conversiones seguidas.** Con la respuesta s entre cada una, convertí 0, 1, 2 y 3. Resultados: 0 → 0, 1 → 1, 2 → 10 y 3 → 11. Con esto comprobé que el 0 se convierte en 0 y que el programa se repite sin reiniciarse.

Parte 2: Bienvenidos a la Cabaña del Misterio

Actividad 2.1 - Programa final para la Cabaña del Misterio

Estrategia para el registro del cliente. Partí de CotizadorPines.java de la Práctica 5 y creé CabanaDelMisterio.java, con todo el código dentro de main. Al inicio pido los cinco datos en este orden, cada uno dentro de su propio ciclo while (true) que solo termina con break cuando el dato es válido:

- **Nombre.** Es el primer dato, leído con nextLine(), para evitar el problema del salto de línea que deja nextInt(). Se rechaza si queda vacío.
- **Tipo de cliente.** Se lee con nextLine(), se limpia con trim() y toUpperCase() y se acepta solo si es un único carácter P, E o R. Con él se guarda el nombre del tipo (Preferido, Estandar o Riesgoso).
- **Capital y tasa.** Con hasNextDouble() compruebo que sea un número; si no lo es, descarto el texto con sc.next() y muestro lo que escribió el usuario. Después de nextDouble() hago un nextLine() para limpiar el renglón. El capital debe ser mayor que cero y la tasa no puede ser negativa.
- **Plazo.** Sigue el mismo esquema, pero exigiendo un número entero mayor que cero.

En todos los casos se imprime un mensaje que explica qué estuvo mal antes de volver a pedir el dato.

Estrategia para el menú. Después del registro, un ciclo while imprime el menú y lee la opción. Si lo escrito no es un entero (hasNextInt() regresa false), el programa descarta el dato, suma un error y avisa. Si es un entero fuera del rango de 0 a 4, también suma un error y avisa. En ambos casos el menú vuelve a aparecer. Si la opción es válida, uso una cadena de if / else if para ejecutarla:

1. **Opción 1.** Muestra los cinco datos del cliente, el interés simple y el total a pagar.
2. **Opción 2.** Muestra los datos del cliente, el valor del préstamo mes por mes con un ciclo, el interés compuesto y el total a pagar.
3. **Opción 3.** Muestra ambos intereses, la diferencia entre ellos y cuál es mayor.
4. **Opción 4.** Pide un nuevo capital con la misma validación del registro (número, mayor que cero) y lo guarda en la variable capital, por lo que todas las cotizaciones posteriores usan el valor nuevo.
5. **Opción 0.** Termina el programa.

Como en la Práctica 5, la tasa se ajusta según el tipo de cliente: al Preferido se le resta 2 puntos (sin bajar de 0), al Estándar no se le cambia y al Riesgoso se le suman 4. El interés simple se calcula como capital × tasa × plazo / 12, y el compuesto con la tasa mensual (tasa / 12) elevada al número de meses con Math.pow.

Estrategia para las cuentas y para terminar el programa. Uso tres variables: operaciones (operaciones efectivas), errores (errores del menú) y motivoFin (un String). operaciones sube en 1 solo cuando se ejecuta una opción de la 1 a la 4, y errores sube solo con las entradas inválidas del menú; los datos inválidos del registro y de la opción 4 se vuelven a pedir en sus propios ciclos y no cuentan. El ciclo del menú se repite mientras no se haya elegido salir, operaciones sea menor que 5 y errores sea menor que 3, por lo que la condición del ciclo cubre los tres casos de término:

1. **Opción 0.** Una bandera salir cambia a true y se guarda el motivo.
2. **Quinta operación efectiva.** Después de mostrar su resultado, si operaciones llega a 5, se guarda el motivo y se avisa que se alcanzó el límite de operaciones.
3. **Tercer error.** Al sumar el error, si errores llega a 3, se guarda el motivo, se avisa el límite de errores y se sale del ciclo con break.

Al salir del ciclo se imprime el resumen final con el nombre del cliente, las operaciones efectivas, los errores del menú, el capital con el que terminó el préstamo y el motivo de término, y después el programa se despide.

Verificación con los datos de Robbie (Robbie Valentino, estándar, $12,899, 15 %, 21 meses). Mi programa muestra la coma como separador decimal por la configuración regional de la terminal, pero los valores son los esperados:

Concepto	Esperado	Obtenido
Interés simple	3385.99	3385,99
Total con interés simple	16284.99	16284,99
Mes 1 (compuesto)	13060.24	13060,24
Mes 21 (compuesto)	16743.71	16743,71
Interés compuesto	3844.71	3844,71
Diferencia entre intereses	458.72	458,72

Los datos coinciden, y el programa indica que el interés compuesto es el mayor. Para el capital de $18,499 de la opción 4, al aplicar las mismas fórmulas obtengo interés simple de 4855.99 (total 23354.99), interés compuesto de 5513.86 (total 24012.86) y una diferencia de 657.87, que también coinciden con los valores esperados.

Ejecuciones.

1. **Robbie.** Registré a Robbie Valentino con los datos anteriores y usé las opciones 1, 2 y 3, cuyos resultados coinciden con la tabla. Después usé la opción 4 para cambiar el capital a $400 y salí con la opción 0. El resumen mostró 4 operaciones efectivas, 0 errores en el menú, un capital final de 400.00 y como motivo que el usuario eligió salir.
2. **Yo mismo (laptop de la Práctica 2).** Me registre a mi mismo con mi nombre y la laptop random de la practica 2 y me puse cliente P, con una tasa del 5% a 12 meses, Despues con la opción 3 compare el interés compuesto y el interés simple y con 5 operaciones cerró el programa.
3. **Cliente ficticio.** Para el cliente ficticio pusie un nombre random (en este caso Panfilo)

Actividad 2.2 - Conclusión y retrospectiva

1. Conclusión de la práctica. Esta práctica me dejó ver la diferencia entre un programa que funciona con los datos que yo escribo en el código y uno que de verdad puede usar otra persona. Lo más importante que

aprendí fue a usar Scanner con hasNextInt() y hasNextDouble() para revisar la entrada antes de leerla, a descartar con next() lo que el usuario escribe mal para no caer en un ciclo infinito, y a combinar ciclos, banderas y contadores para controlar cuándo termina un programa. Las complicaciones más claras fueron el salto de línea que queda después de leer un número (por eso pedí el nombre primero y agregué nextLine() después de cada número), un error de compilación por un punto y coma que faltaba al declarar el capital, y un error al ejecutar desde la carpeta bin por escribir mal el nombre de la clase. Los tres me enseñaron a leer con calma los mensajes del compilador.

2. Desarrollo del proyecto de la Cabaña del Misterio. El proyecto empezó con una cotización básica con datos escritos en el código (Práctica 2). En la Práctica 4 el programa ganó variables y salida con formato (printf), y en la Práctica 5 aprendió a tomar decisiones: ajustar la tasa según el tipo de cliente, calcular el interés simple y el compuesto, y rechazar plazos negativos. En esta práctica el programa dejó de depender de mí: ahora pide los datos, los valida, ofrece un menú, permite cambiar el capital y se protege con límites de operaciones y de errores. Siento que el avance fue de escribir cálculos sueltos a construir un programa completo con el que otra persona puede trabajar sin conocer el código.

3. Encontrar errores con todo en main y partes repetidas. Con todo el programa dentro de main (más de 250 líneas), encontrar y corregir errores fue más lento: tenía que subir y bajar por el archivo para ubicar un problema, y las llaves y los ciclos anidados hacían fácil perderme. Hay varias partes muy parecidas entre sí: los ciclos de validación del capital, la tasa y el plazo, que siguen casi el mismo patrón; la validación del nuevo capital de la opción 4, que repite la del registro; los dos avisos de error del menú (entrada que no es número y número fuera de rango); y la impresión de los datos del cliente en las opciones 1 y 2. Si pudiera reutilizar esas partes en bloques separados, el código sería más corto y más fácil de corregir.